

TP integrador – Repetitivas- Condicionales y Secuenciales.

Ejercicio 1— “Caja del Kiosco”

Objetivo: Simular una compra con validaciones y cálculo de total.

Requisitos

1. Pedir **nombre del cliente** (solo letras, validar con `.isalpha()` en `while`).
2. Pedir **cantidad de productos a comprar** (número entero positivo, validar con `.isdigit()` en `while`).
3. Por cada producto (usar `for`):
 - Pedir **precio** (entero, validar `.isdigit()`).
 - Pedir **si tiene descuento** S/N (validar con `while`, aceptar `s` o `n` en cualquier mayuscula/minuscula).
 - Si tiene descuento: aplicar **10%** al precio de ese producto.
4. Al final mostrar:
 - Total sin descuentos
 - Total con descuentos
 - Ahorro total
 - Promedio por producto (usar `float` y formatear con `:.2f`, ejem:

```
x = 3.14159
```

```
print(f"{x:.2f}")
```

Validaciones obligatorias

- Sin `try/except`.
- No aceptar vacío en nombre (si queda vacío, es error).
- Cantidad > 0 (si ingresa 0, volver a pedir).

Salida esperada (ejemplo)

```
Cliente: Ana
Cantidad de productos: 3
Producto 1 - Precio: 100 Descuento (S/N): s
Producto 2 - Precio: 50 Descuento (S/N): n
Producto 3 - Precio: 200 Descuento (S/N): s

Total sin descuentos: $350
Total con descuentos: $320.00
Ahorro: $30.00
Promedio por producto: $106.67
```

Ejercicio 2 — “Acceso al Campus y Menú Seguro”

Objetivo: Login con intentos + menú de acciones con validación estricta.

Requisitos

1. Definir credenciales fijas en el código:
 - o usuario correcto: "alumno"
 - o clave correcta: "python123"
2. Permitir **máximo 3 intentos** para ingresar usuario y clave.
3. Si falla 3 veces: mostrar “Cuenta bloqueada” y terminar.
4. Si ingresa bien: mostrar un menú repetitivo (usar `while`) hasta elegir salir:
 1. Ver estado de inscripción (mostrar “Inscripto”)
 2. Cambiar clave (pedir nueva clave y confirmación; deben coincidir)
 3. Mostrar mensaje motivacional (1 frase)
 4. Salir
5. Validación del menú:
 - o Debe ser número (`.isdigit()`)
 - o Debe estar entre 1 y 4

Cambio de clave

- La nueva clave debe tener **mínimo 6 caracteres** (validar con `len()`), si no, rechazar.

Salida esperada

```
Intento 1/3 - Usuario: alumno
Clave: xxx
Error: credenciales inválidas.
```

```
Intento 2/3 - Usuario: alumno
Clave: python123
Acceso concedido.
```

```
1) Estado  2) Cambiar clave  3) Mensaje  4) Salir
Opción: a
Error: ingrese un número válido.
Opción: 5
Error: opción fuera de rango.
```

Opción: 2
Nueva clave: 123
Error: mínimo 6 caracteres.
...

☒ Ejercicio 3 (Alta) — “Agenda de Turnos con Nombres (sin listas)”

Contexto

Hay 2 días de atención: **Lunes** y **Martes**.
Cada día tiene cupos fijos:

- Lunes: 4 turnos
- Martes: 3 turnos

Reglas

1. Pedir nombre del operador (solo letras).
2. Menú repetitivo hasta salir:
 1. Reservar turno
 2. Cancelar turno (por nombre)
 3. Ver agenda del día
 4. Ver resumen general
 5. Cerrar sistema
3. **Reservar:**
 - Elegir día (1=Lunes, 2=Martes).
 - Pedir nombre del paciente (solo letras).
 - Verificar que **no esté repetido en ese día** (comparando con las variables ya cargadas).
 - Guardar en el **primer espacio libre** (ej. lunes1, lunes2...).
4. **Cancelar:**
 - Elegir día.
 - Pedir nombre del paciente (solo letras).
 - Si existe, cancelar y dejar el espacio vacío ("").
5. **Ver agenda del día:**

- Mostrar los turnos del día en orden (Turno 1..N), indicando “(libre)” si está vacío.
6. **Resumen general:**
- Turnos ocupados y disponibles por día.
 - Día con más turnos (o empate).

Restricciones

- ✗ No listas, no diccionarios, no sets, no tuplas.
- ☒ Se permite usar "" como “vacío”.
- ☒ Validaciones con `.isalpha()` y `.isdigit()` (sin `try/except`).

Ejercicio 4 — “Escape Room: La Bóveda”

Historia

Sos un agente que intenta abrir una bóveda con **3 cerraduras**. Tenés **energía** y **tiempo** limitados.

Si abrís las 3 cerraduras **antes** de quedarte sin energía o sin tiempo, ganás.

Variables iniciales (NO se piden por teclado)

- `energia = 100`
- `tiempo = 12`
- `cerraduras_abiertas = 0`
- `alarma = False`
- `codigo_parcial = ""`

Validaciones obligatorias

- No usar `try/except`.

- Pedir **nombre del agente** y validar con `.isalpha()` en un `while`.
- Validar **opciones del menú** y cualquier número pedido con `.isdigit()` en un `while`.
- El juego debe funcionar con **estructuras secuenciales, condicionales y repetitivas** (puede usar funciones propias del lenguaje como `.lower()`, `len()`, `formato`, etc.).

Regla anti-spam (muy importante)

Para evitar que el jugador gane eligiendo “**Forzar cerradura**” 3 veces seguidas al iniciar:

- ☒ Si el jugador elige **Forzar cerradura (opción 1)** 3 veces seguidas, entonces:
- se cobra el costo normal (-20 energía, -2 tiempo),
 - **NO abre cerradura**, y
 - se activa la alarma automáticamente (`alarma = True`) porque “la cerradura se trabó”.

Si el jugador elige opción 2 o 3, se corta la racha de “forzar seguidas”.

Menú de acciones (se repite mientras el juego siga)

El juego continúa mientras:

- `energia > 0, tiempo > 0, cerraduras_abiertas < 3`
- y **no esté bloqueado** por alarma.

En cada turno mostrar el estado y el siguiente menú:

1. **Forzar cerradura** (*costo: -20 energía, -2 tiempo*)
 - Si la energía está por debajo de 40, hay “riesgo de alarma”:
 - pedir un número 1-3 (validado). Si elige 3 → `alarma=True`.
 - Si no hay alarma, abre 1 cerradura.
 - **Regla anti-spam**: si es la 3ra vez seguida forzando, se activa alarma y no abre.
2. **Hackear panel** (*costo: -10 energía, -3 tiempo*)
 - Debe usar un `for` de 4 pasos mostrando progreso.
 - En cada paso sumar una letra al `codigo_parcial` (por ejemplo “A”).
 - Si `len(codigo_parcial) >= 8`, se abre automáticamente 1 cerradura si todavía faltan.

3. **Descansar** (costo: +15 energía (máx 100), -1 tiempo; si alarma ON: -10 energía extra)
-

Regla de bloqueo por alarma

- Si `alarma == True` y `tiempo <= 3` y todavía no se abrió la bóveda, el sistema se bloquea y se pierde.
-

Condiciones de fin

- Si `cerraduras_abiertas == 3` → **VICTORIA**
- Si `energia <= 0` o `tiempo <= 0` → **DERROTA**
- Si se bloquea por alarma → **DERROTA (bloqueo)**

Ejercicio 5 — “Escape Room:”La Arena del Gladiador”

1. Descripción del Escenario

Vas a desarrollar un simulador de batalla por turnos en Python. El programa enfrentará a un usuario (Gladiador) contra un oponente controlado por la computadora (Enemigo). El objetivo es reducir los puntos de vida del oponente a cero antes de que él lo haga contigo. Este ejercicio evalúa el uso de variables (`int`, `float`, `string`, `boolean`), estructuras de control (`if/elif/else`), ciclos (`while` y `for`) y validación de datos estricta.

2. Requerimientos Técnicos

A. Tipos de Datos

Debes utilizar obligatoriamente los siguientes tipos de datos para las variables del juego:

- **String:** Para el nombre del jugador.
- **Int:** Para los Puntos de Vida (HP) y cantidad de pociones.
- **Float:** Para el cálculo del daño (ej: un golpe crítico multiplica el ataque por 1.5).

- **Boolean:** Para controlar si el juego sigue activo o quién tiene el turno.

B. Reglas de Validación (!Importante!)

- **No está permitido** usar bloques `try / except`.
- Para validar texto, debes usar el método `.isalpha()` dentro de un ciclo `while`.
- Para validar números, debes usar el método `.isdigit()` dentro de un ciclo `while`.

3. Flujo del Programa

Paso 1: Configuración del Personaje

El programa inicia pidiendo el nombre del Gladiador.

- **Validación:** El nombre solo puede contener letras. Si el usuario ingresa números, símbolos o lo deja vacío, el programa debe decir "Error: Solo se permiten letras" y volver a preguntar hasta que sea válido.

Paso 2: Inicialización de Estadísticas

El programa debe definir las variables iniciales (sin preguntar al usuario):

- Vida del Gladiador: 100 (int)
- Vida del Enemigo: 100 (int)
- Pociones de Vida: 3 (int)
- Daño base "Ataque Pesado": 15 (int)
- Daño base del enemigo: 12 (int)
- Turno Gladiador : True (booleano)

Paso 3: El Ciclo de Combate

El juego entra en un ciclo que se repite **mientras** ambos combatientes tengan más de 0 puntos de vida.

Turno del Jugador:

Muestra la vida actual de ambos y las pociones restantes. Luego, ofrece un menú con 3 opciones:

1. Ataque Pesado
2. **Ráfaga Veloz** (Requiere uso de `for`)
3. Curar

• **Validación del Menú:** El programa debe pedir la opción al usuario. 1. Verificar que lo ingresado sea un número (`.isdigit()`).

2. Verificar que el número sea **1, 2 o 3**.

- Si falla alguna validación, mostrar mensaje de error y volver a pedir.

Lógica de las Acciones:

Acción A: Ataque Pesado (Opción 1)

- Calcula el daño final. Si la vida del enemigo es menor a 20 puntos, el jugador realiza un "Golpe Crítico" multiplicando su daño base por 1.5 (resultado float).
- Resta el daño a la vida del enemigo.
- Muestra un mensaje: *"¡Atacaste al enemigo por X puntos de daño!"*

Acción B: Ráfaga Veloz (Opción 2)

- Esta acción realiza una serie de golpes rápidos. **Debes implementar un bucle `for`**.
- El bucle debe repetirse **3 veces** (usando `range`).
- Dentro del bucle, en cada repetición: 1. Resta **5 puntos** de daño a la vida del enemigo.
- 2. Muestra el mensaje: *" > Golpe conectado por 5 de daño "*.
-

Acción C: Curar (Opción 3)

- Si tienes pociones (> 0): Suma 30 puntos a tu vida y resta 1 poción.
- Si **NO** tienes pociones: Muestra "*¡No quedan pociones!*" y pierdes el turno (el enemigo ataca igual).

Turno del Enemigo:

Justo después de tu acción, el enemigo ataca automáticamente.

- Resta el daño base del enemigo (12) a tu vida.
- Muestra un mensaje: "*¡El enemigo te atacó por 12 puntos de daño!*"

Paso 4: Fin del Juego

Cuando el ciclo termine (porque la vida de alguno llegó a 0 o menos), debes evaluar:

- Si `vida_jugador > 0`: Mostrar "*¡VICTORIA! [Nombre] ha ganado la batalla.*"
- Si `vida_jugador <= 0`: Mostrar "*DERROTA. Has caído en combate.*"

4. Ejemplo de Ejecución (Consola)

Plaintext

```
--- BIENVENIDO A LA ARENA ---
Nombre del Gladiador: Leonidas1
Error: Solo se permiten letras.
Nombre del Gladiador: Leonidas
=== INICIO DEL COMBATE ===
Leonidas (HP: 100) vs Enemigo (HP: 100) | Pociones: 3
Elige acción:
1. Ataque Pesado
2. Ráfaga Veloz
3. Curar
Opción: A
Error: Ingrese un número válido.
Opción: 2
>> ¡Inicias una ráfaga de golpes!
> Golpe conectado por 5 de daño
> Golpe conectado por 5 de daño
> Golpe conectado por 5 de daño
>> ¡El enemigo contraataca por 12 puntos!
=== NUEVO TURNO ===
Leonidas (HP: 88) vs Enemigo (HP: 85) | Pociones: 3
...
```